

Excel Agent 可视化操作工具

开发文档 · MVP · 技术路线讲解

让 AI Agent 通过「语义化 API + CLI + Skill」驱动 Excel，
完成写入、格式化、公式、图表、导出等简单可视化操作，并实时看到界面变化。

10
MVP 功能

6
核心模块

COM
技术路线

技术路线：Windows COM/OLE 自动化 (pywin32) · 项目代号 excel-agent (MVP)

目 录

01

可行性

COM 自动化原理可推广，为何选 Excel

02

MVP 功能清单

10 项核心能力：写入 / 样式 / 公式 / 图表 / 导出

03

架构与代码实现

backend → ExcelDoc → quality → CLI → Skill 全链路

04

端到端示例与工程化

example.py · 资源回收 · 测试验证 · 扩展方向

可行性：原理可推广

harness-anything 驱动 WPS 的本质：调用软件暴露的 COM 自动化对象模型

Python —pywin32—► COM 接口 —► 软件进程 —► 界面/文档

只要软件对外暴露 COM/OLE 对象模型，即可用相同模式驱动：

软件	COM 入口	典型操作对象
Excel	Excel.Application	工作簿/单元格/公式/图表
Word	Word.Application	文档/段落/表格/样式
PowerPoint	PowerPoint.Application	幻灯片/形状/动画
AutoCAD	AutoCAD.Application	图形实体/图层/标注
MATLAB	matlab.application	矩阵/绘图/仿真

结论：可行。选 Excel —— 最常用 · COM 自动化最成熟 · 可视化价值高 · 与 WPS 表格同源

MVP 功能清单（简单即可）

#	功能	API
1	新建/打开工作簿	new() / open(path)
2	写入单元格/整行/矩阵	write() / write_row() / write_matrix()
3	合并单元格	merge("A1:C1")
4	样式（加粗/字号/颜色/对齐）	style(...)
5	列宽行高	col_width() / row_height()
6	公式	formula("D2","=SUM(...)")
7	生成图表	add_chart(...)
8	图表导出 PNG	export_chart_png()
9	保存 xlsx / 导出 PDF	save() / export_pdf()
10	回收进程	close()

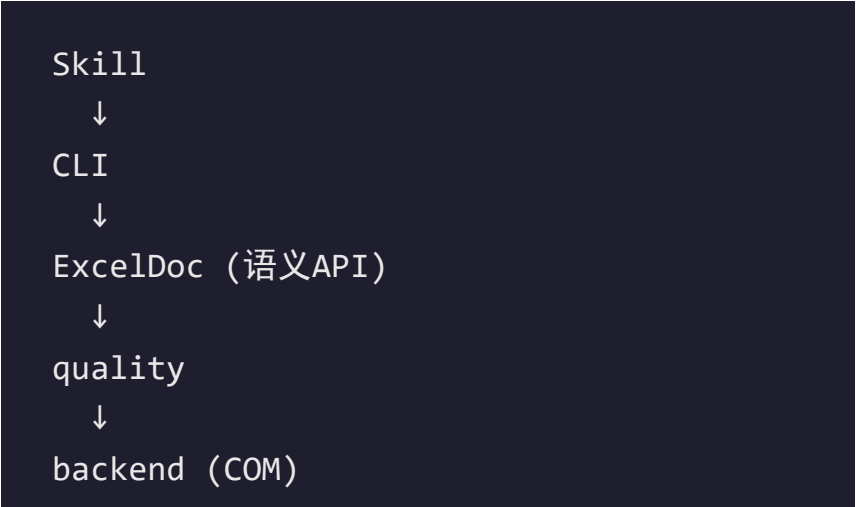
全部「语义化 API」：Agent 说需求，代码驱动 Excel 实时可视化执行

架构与目录

excel-agent 项目 · 六模块 七文件

模块	职责
backend.py	COM 后端 (唯一 import win32com)
core/excel_doc.py	语义化 API (ExcelDoc)
core/quality.py	简单质量检查
cli.py	命令行入口
example.py	端到端示例
skills/SKILL.md	Agent 技能定义

分层调用链



核心原则：
只有 backend.py import win32com，
上层与 COM 完全解耦。

后端层 · backend.py

设计要点

```
# backend.py COM 后端
```

```
import os, pythoncom
```

```
import win32com.client as win32
```

```
FILE_FORMAT = {"xls": 51,  
               "xls": 56, "csv": 6}
```

```
EXPORT_FORMAT = {"pdf": 0, "xps": 1}
```

```
class ExcelBackend:
```

```
    def __init__(self, visible=True):  
        pythoncom.CoInitialize()  
        self.app = win32.Dispatch(  
            "Excel.Application")  
        self.app.Visible = visible  
        self.app.DisplayAlerts = False  
        self._workbook = None
```

```
    def save(self, path, fmt):  
        self._workbook.SaveAs(path,  
                               FileFormat=FILE_FORMAT[fmt])  
    def export_pdf(self, path):  
        self._workbook.ExportAsFixedFormat(  
            path, EXPORT_FORMAT["pdf"],
```

常量映射

避免魔法数字

可视化开关

Visible=True 实时看到

Excel 界面变化

DisplayAlerts=False 防弹窗

生命周期

close() 统一回收进程

pythoncom.CoInitialize()

线程安全

后端层 = 软件进程的唯一「接入口」：其余模块一律经由它中转

语义 API 层 · core/excel_doc.py

数据写入

write(cell, v)
write_row(start, vals)
write_matrix(start, m)
formula(cell, expr)

格式化

style(bold/size/color/...)
merge("A1:C1")
col_width / row_height

图表

add_chart(rng, type)
column/line/pie/bar
export_chart_png()

生命周期

new() / open(path)
save() / export_pdf()
close() 回收进程

常量设计

rgb(r,g,b) → BGR 长整型
HALIGN / VALIGN
CHART_TYPE 映射表

链式调用

每个方法 return self
new().write().style()
一步到底

Excel COM 颜色为长整型(BGR): $rgb = r + (g < < 8) + (b < < 16)$ · 图表类型: column=51 / line=4 / pie=5 / bar=57

质量检查 + CLI 入口

core/quality.py

```
# quality.py
def check_empty_used_range(doc):
    """检查已用区域内空单元格"""
    used = doc.sheet.UsedRange
    problems = []
    for r in range(1, used.Rows.Count+1):
        for c in range(1,
            used.Columns.Count+1):
            if used.Cells(r,c).Value in
                (None, ""):
                problems.append(
```

check_column_too_narrow(doc, cols, max_width=8)

检测列宽过窄导致的「显示截断」问题

cli.py · 参数一览

参数	作用
--new	新建工作簿
--open PATH	打开已有文件
--write CELL VALUE	写单元格
--formula CELL EXPR	写公式
--chart RANGE	生成图表
--save PATH	保存 xlsx
--pdf PATH	导出 PDF
--invisible	后台运行不弹窗

一行命令 = 一次 Agent 调用：CLI 是 Skill 与 ExcelDoc 之间的「桥」

Agent 技能定义 · skills/SKILL.md

Skill = Agent 的操作手册

```
---
name: excel-agent
description: 驱动 Excel 完成写入/
  格式化/公式/图表/导出等简单
  可视化操作
frontmatter
---
```

```
name: excel-agent
## 能力
- 新建/打开工作簿，写入数据
- 合并、样式、公式、图表
- 保存 xlsx、导出 PDF / PNG
```

调用方式

python example.py

python cli.py --new --write A1 值

调用方式

example.py 演示

cli.py 命令行

两种接入路径

能力清单

写入 / 格式化

公式 / 图表 / 导出

对齐 MVP 10 项功能

工程约定

Visible=True 可视化

close() 必回收

可被 AI Agent 自动调用

有了 SKILL.md, AI Agent 就能「读懂并驱动」excel-agent 工具

端到端示例 · example.py

运行效果 · 可视化全程

```
# example.py
doc = ExcelDoc(visible=True)
doc.new("销售报表")

# 表头 + 合并 + 样式
doc.merge("A1:D1").write("A1",
    "季度销售报表")
doc.style("A1:D1", bold=True,
    size=16, bg=(31,78,121),
    color=(255,255,255),
    halign="center")
# 数据 + 公式
doc.write_row("A2", [季度,产品A,
    产品B,合计])
doc.write_matrix("A3", [...])
for r in range(3,7):
    doc.formula(f"D{r}",
        f"=SUM(B{r}:C{r})")

# 图表 + 导出
```

实时可见

即时反馈

数据还格式

表头变色加粗

三类产物

销售报表.xlsx

销售报表.pdf

chart.png

公式自动求和

=SUM(B:C) 逐行计算

合计列自动填充

图表同步生成

进程回收

close(save=True) 一次运行，完整演示

数据已保存 「新建 → 写数 → 美化 → 公式 → 图表 → 导出」

进程不残留 全流程

example.py 是 Skill 的「活文档」：跑一遍 = 看一遍全部能力

资源回收与错误处理

问题	对策
进程残留	close() 必须放在 finally 中; DisplayAlerts=False 避免卡弹窗
脚本中途崩溃	用上下文管理器包装, 异常也自动回收
多线程 COM 冲突	__init__ 里调用 pythoncom.CoInitialize()

推荐的上下文管理器写法（推荐工程用法）

```
class ExcelDoc:
    def __enter__(self): return self
    def __exit__(self, *a): self.close()

with ExcelDoc() as doc:           # 离开 with 自动 close()
```

测试验证 + 扩展方向

测试验证清单

扩展方向

同原理迁移

- [x] example.py 弹出 Excel 并生成三个产物文件
- [x] --invisible 后台运行不弹窗
- [x] 反复运行后任务管理器无残留 EXCEL.EXE
- [x] export_pdf 内容正确、图表完整
- [x] 中文写入不乱码

多软件统一

抽象 Backend 接口
Excel/Word/Ppt 同一套
write/style/export 契约

非 COM 软件

浏览器用 CDP
GUI 用 UI Automation
(pywinauto)

统一 Skill

一个 Skill 通吃
Office 全家桶
Agent 随调随用

质量扩展

更多检查项
数据一致性校验
报表模板校验

云端化

本地 COM + 远程 API
Agent 平台接入
Web 可视化

思路核心：接口驱动而非模拟点击 —— 同一模式可复制到所有带接口的软件

总结 · 一条完整的自动化链路

AI Agent（说需求）

↓ SKILL.md（教怎么做）

↓ cli.py（命令行）

可行性

COM 自动化

原理可推广

Excel 最成熟

MVP 交付

10 项功能

6 大模块

开箱即用

可视化

Visible=True

操作实时可见

教学演示友好

Agent 化

SKILL.md 技能包

CLI 一行调用

工程化

with 自动回收

质量检查

可扩展

同原理迁移

Office 全家桶

excel-agent = 用 100 行代码，让 Agent 亲手操作 Excel 的完整范本

谢 谢

提问与交流

「接口驱动而非模拟点击」

让 AI 亲手操作 Excel，实时看见每一次变化。

Excel-Agent 可视化操作工具 · 开发文档 · 技术路线 COM/pywin32